

Customer Feedback System – Frontend (React / Angular) Addendum

1. SPA Responsibilities

Provide customer and store manager user experiences: self-registration, login, feedback submission, dashboard summary (KPIs, trends, distribution), and role-aware navigation. Handle JWT, client-side validation, accessible UI, and error reporting.

2. Technology Choices

React: React Router for routing; Redux Toolkit or Zustand for state; Material UI for components; React Hook Form for forms; Axios/Fetch for API; Chart.js/Recharts for charts. Angular: Angular Router; NgRx for state; Angular Material; Reactive Forms; HttpClient; ngx-charts/Chart.js.

3. Authentication & Session

Access token stored in memory (or secure storage with rotation); refresh token via secure, httpOnly cookie set by server. Use an Axios/HttpClient interceptor to attach JWT, handle 401→refresh→retry; auto-logout on repeated failures. Protect manager routes via guards.

4. Routing & Guards

Public routes: Register, Login, Submit Feedback (optionally anonymous). Protected routes: Manager Dashboard, Feedback List, Category Management. Implement route guards (React: wrapper components; Angular: CanActivate) that verify role claims.

5. State Management

Use a thin global store for auth/session and lightweight local state for forms and filters. Persist selected store context. Side-effects via RTK Query (React) or NgRx Effects (Angular).

6. Forms & Validation

React: React Hook Form + Zod/Yup; Angular: Reactive Forms + custom validators. Mirror server rules; sanitize inputs; constrain comment length; rating 1–5. Show inline errors and disable submit until valid.

7. UI / Accessibility / i18n

Material design components, semantic HTML, ARIA labels, keyboard navigation, color-contrast. Optional i18n using react-intl or Angular i18n. Responsive layout; skeleton loaders; empty states.

8. Error Handling & Observability

Global error boundary (React) / global error handler (Angular). Map API ProblemDetails to toast/banner messages. Client telemetry: page views, API timings, error rates; send structured logs to backend (without PII).

9. Security (Frontend)

CSP headers, XSS prevention; sanitize any HTML (e.g., DOMPurify) before rendering; avoid storing tokens in localStorage when possible; use SameSite cookies; strict CORS settings; prevent

open redirects.

10. Performance

Lazy-loaded routes (Angular modules / React code-splitting), tree-shaking, memoization, virtualization for long lists, client-side caching of categories, CDN hosting, gzip/brotli, image optimization.

11. Testing (Frontend)

React: Jest + React Testing Library; Angular: Jasmine/Karma or Jest; E2E: Playwright/Cypress for login, submission, dashboard; accessibility checks via axe-core. Integrate with CI.

12. CI/CD (Frontend)

Lint (ESLint/TSLint), unit tests, build optimized artifacts, upload to CDN/Blob Storage behind WAF/CDN; cache-busting via content hashes; invalidate CDN; feature flags controlled via config.

13. Suggested Folder Structures

React (example)

```
/ src
/ app
  / components
  / pages
    - Login.tsx
    - Register.tsx
    - FeedbackForm.tsx
    - Dashboard.tsx
  / features
    / auth
    / feedback
    / reporting
  / services
    - apiClient.ts
    - feedbackApi.ts
  / store
    - index.ts
  / routing
    - AppRoutes.tsx
  / hooks
  / styles
```

Angular (example)

```
/ src
/ app
  / core
    - interceptors
    - guards
    - services
  / shared
    - components
    - directives
    - pipes
  / features
    / auth
    / feedback
    / reporting
  / pages
    - login
    - register
```

- feedback-form
- dashboard

```

/ state (NgRx)
- reducers
- effects
/ app-routing.module.ts

```

14. API Client Patterns (TypeScript)

```

// apiClient.ts
import axios from 'axios';

export const api = axios.create({ baseURL: process.env.REACT_APP_API_BASE });

api.interceptors.request.use((config) => {
  const token = window.sessionStorage.getItem('accessToken');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

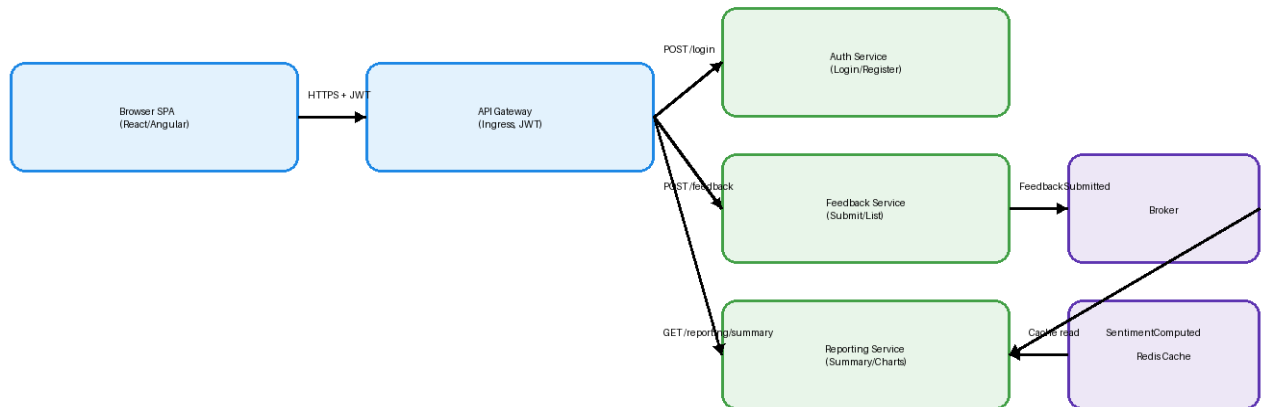
api.interceptors.response.use(
  (resp) => resp,
  async (error) => {
    if (error.response?.status === 401) {
      // try refresh
      await axios.post('/api/auth/refresh', { refreshToken: 'cookie' });
      // retry original request (simplified)
      return api.request(error.config);
    }
    return Promise.reject(error);
  }
);

// feedbackApi.ts
export async function submitFeedback(payload: {
  storeId: string; rating: number; comment?: string; categoryId?: string; metadata?: any;
}) {
  const { data } = await api.post('/api/feedback', payload);
  return data;
}

```

15. SPA Data Flow Diagram

SPA (React/Angular) Data Flow



Legend: SPA → API Gateway (JWT) → Auth/Feedback/Reporting; FeedbackPublished events via Broker → Analytics; Redis caches reporting reads.